

`data/eval_dataset_candidates.csv` 不是人工整理出来的，它是我把 `[data/eval_dataset_candidates.json]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/data/eval_dataset_candidates.json`) 扁平化导出的 CSV。
真正的生成逻辑在 `[app/evaluation/build_test_set.py]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py`)。

核心流程是这样的：

1. 先读药物表

来源是 `[match_data_preprocessing/data/enhanced_drug_table_v1_structured.csv]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/match_data_preprocessing/data/enhanced_drug_table_v1_structured.csv`)，在 `load_drug_table()` 里做了几件事：

- 解析 `disease_key_list` 和 `symptom_list`
 - 算药物排序分：`avg_rating * log1p(total_reviews)`
 - 去掉没有 `disease key` 的药
 - 去掉 `disease` 全是 `others` 的药
- 代码在 `[build_test_set.py]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py:62`)

2. 再读 query 来源

来源不是 LLM 现生成，而是直接复用 `[app/dataset_module/drugs_training_dataset/eval_dataset_llm_v2.json]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/dataset_module/drugs_training_dataset/eval_dataset_llm_v2.json`)。在 `load_query_rows()` 里：

- 抽出 `diseases`
 - 抽出 `symptoms`
 - 去掉 `disease` 里的 `others`
 - 优先保留原始 `sentence` 作为 `symptom_text`
- 代码在 `[build_test_set.py]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py:75`)

3. 按 disease 数量分桶抽样

脚本固定想生成 300 条：

- 150 条 `single-disease`
 - 100 条 `double-disease`
 - 50 条 `triple-plus`
- 这是在 `main()` 里定死的默认参数，然后由 `build_query_buckets()` 和 `collect_bucket_cases()` 去凑齐。代码在 `[build_test_set.py]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py:205`) 和 `[build_test_set.py]` (`/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py:288`)

4. 给每个 query 选 relevant drugs

这一步是关键，在 `select_relevant_drugs()`：

- 只从该 `disease` 对应的药池里选
- 如果 `query` 有症状，药物 `matched_symptoms` 必须和 `query` 症状有交集
- 每个 `disease` 最多取 3 个药
- 总数最多 9 个，最少必须有 3 个，否则这条 `candidate` 丢掉

- 排序优先看：
 - 症状重叠数
 - 药物 score
 - 药物覆盖 disease 数
 代码在 [build_test_set.py](/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py:126)

5. 生成最终 candidate case

在 query_to_candidate() 和 finalize_case() 里组装成：

- query_id
- symptom_text
- diseases
- symptoms
- relevant_drugs
- relevance_scores

其中：

- symptom_text 优先用 eval_dataset_llm_v2.json 里的原句
 - 没有原句时才用模板句补
- 代码在 [build_test_set.py](/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py:180) 和 [build_test_set.py](/Users/jayden/Desktop/7012 datamining and text/project_march/ARIN7102_Group_Project/app/evaluation/build_test_set.py:160)

所以一句话概括：

```
eval_dataset_candidates.csv =
从 eval_dataset_llm_v2.json 抽 query,
从 enhanced_drug_table_v1_structured.csv 按 disease/symptom 匹配并按评分选药,
按 150/100/50 分桶采样生成的 300 条候选集,
然后我再把这个 JSON 扁平化成 CSV。
```

如果你要，我下一步可以直接帮你画出一条 candidate 的“来源链”：

```
sentence -> diseases/symptoms -> drug pool -> relevant_drugs
```

这样你能更直观看出每一列是怎么来的。